



รายงาน

ชื่อ

Painner

เสนอ

ผศ.ดร.ฐาปนา บุญชู

จัดทำโดย

กลุ่มที่ 5

นายจุติณัฐ รัตนมงคลกุล	6609650228
นายภูวฤทธิ์ เจริญพล	6609650590
นายรพีพันธ์ นะราช	6609650624
นายศุภณัฐ ตั้งกิจวุฒิกุล	6609650665
นายสุทธิพนธ์ ประทุมทอง	6609650699
นายสุรบดี ผาสุข	6609650707

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา

คพ.363 โครงการทางวิศวกรรมซอฟต์แวร์

คณะวิทยาศาสตร์และเทคโนโลยี สาขาวิชาวิทยาการคอมพิวเตอร์

มหาวิทยาลัยธรรมศาสตร์ วิทยาเขตศูนย์รังสิต

ภาคเรียนที่ 2 ปีการศึกษา 2568

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ในยุคปัจจุบันที่การทำงานเป็นทีมมีความซับซ้อนมากขึ้น การประสานงานและติดตามความก้าวหน้าของงานจึงเป็นสิ่งจำเป็นอย่างยิ่ง ทีมพัฒนาซอฟต์แวร์และองค์กรต่าง ๆ มักประสบปัญหาในการแบ่งงาน ติดตามสถานะ และสื่อสารภายในทีม ซึ่งส่งผลให้เกิดความล่าช้าและความผิดพลาดในการทำงาน

โครงการ PannerAtlas จึงถูกพัฒนาขึ้นมาเพื่อแก้ปัญหาดังกล่าว โดยสร้างเป็นเว็บแอปพลิเคชันสำหรับบริหารจัดการโปรเจกต์และงานของทีม (Project Management System) ที่รวมฟีเจอร์สำคัญ ได้แก่ การจัดการทีม การสร้างโปรเจกต์ การจัดการงานแบบ Kanban Board และระบบแชทในโปรเจกต์ ไว้ในแพลตฟอร์มเดียว เพื่อให้ทีมสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์

- เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับบริหารจัดการโปรเจกต์และงาน โดยใช้เทคโนโลยี ReactJS, Node.js และ MongoDB
- เพื่อให้ทีมสามารถสร้างและจัดการทีม โปรเจกต์ บอร์ด และงาน (Task) ได้ในระบบเดียว
- เพื่อให้สมาชิกในทีมสามารถสื่อสารกันผ่านระบบ Chat ที่รวมอยู่ในโปรเจกต์
- เพื่อฝึกทักษะการพัฒนาเว็บแอปพลิเคชันแบบ Full-Stack ตามหลัก MVC Architecture และ RESTful API

บทที่ 2 การออกแบบระบบ

2.1 Architecture Overview

ระบบ PannerAtlas พัฒนาตาม MVC Architecture (Model-View-Controller) โดยแบ่งโครงสร้างระบบออกเป็น 3 ส่วนหลัก ได้แก่

- Model — กำหนดโครงสร้างข้อมูลด้วย Mongoose Schema บน MongoDB ครอบคลุม 9 Collections ได้แก่ User, Team, TeamMember, Project, Board, Task, ProjectMember, ProjectChat และ Message
- View — ส่วน Frontend พัฒนาด้วย ReactJS เชื่อมต่อกับ Backend ผ่าน REST API โดยใช้ Axios
- Controller — ประมวลผล Business Logic บน Node.js + Express.js โดยแต่ละ Resource มี Controller แยกต่างหาก

การสื่อสารระหว่าง Client และ Server ใช้รูปแบบ RESTful API โดย Frontend ส่ง HTTP Request พร้อม JWT Token ใน Authorization Header ไปยัง Backend ซึ่งตรวจสอบสิทธิ์ผ่าน Middleware authenticate.js ก่อนส่งต่อให้ Controller ประมวลผล

2.2 เทคโนโลยีที่ใช้

ฝั่ง	เทคโนโลยี	รายละเอียด
Frontend	ReactJS 18	UI Library สำหรับสร้าง Single-Page Application
	React Router DOM 6	จัดการ Routing ระหว่างหน้าต่าง ๆ
	Axios 1.6	HTTP Client สำหรับเรียก REST API
	Vite 5	Build Tool สำหรับ Development และ Production
Backend	Node.js + Express 4	Web Server Framework สำหรับสร้าง REST API
	Mongoose 8	ODM Library เชื่อมต่อ MongoDB

	JWT (jsonwebtoken)	ระบบ Authentication ด้วย Token
	bcryptjs	เข้ารหัส Password ก่อนจัดเก็บ
Database	MongoDB	NoSQL Document Database
Deploy	Docker	Containerize ทั้ง Frontend และ Backend
	Nginx	Serve Static Files สำหรับ Frontend

2.3 การออกแบบฐานข้อมูล

ระบบใช้ MongoDB เป็นฐานข้อมูล โดยออกแบบ Schema ด้วย Mongoose แบ่งเป็น 9 Collections ดังนี้

Collection	Field หลักและความสัมพันธ์
User	firstName, lastName, email (unique), username (unique), password (bcrypt hash), avatarColor
Team	name, icon, createdBy (ref: User)
TeamMember	teamId (ref: Team), userId (ref: User), role (Owner/Manager/Member) — unique index (teamId, userId)
Project	name, teamId (ref: Team), createdBy (ref: User), status (Planning/In Progress/Review/Done)
Board	name (เช่น Sprint 1), projectId (ref: Project), createdBy (ref: User)
Task	title, description, status (todo/inprogress/review/done), category (Frontend/Backend/DevOps/Design/QA), assigneeId (ref: User), projectId, boardId, dueDate, createdBy
ProjectMember	projectId (ref: Project), userId (ref: User), role, displayName, avatarColor — unique index (projectId, userId)
ProjectChat	projectId (ref: Project, unique) — 1 ChatRoom ต่อ 1 Project
Message	chatId (ref: ProjectChat), senderId (ref: User), text (max 2000 chars)

2.4 การออกแบบส่วนต่อประสาน (UI Design)

การออกแบบ UI ใช้รูปแบบ Dark Theme สีน้ำเงินเข้ม โดยมี Layout หลักเป็น Sidebar ด้านซ้าย สำหรับ Navigation และ Main Content Area ด้านขวา แบ่งออกเป็นหน้าต่าง ๆ ดังนี้

- หน้า Login/Register — ฟอรัมเข้าสู่ระบบและสมัครสมาชิก
- หน้า Dashboard — แสดงภาพรวม Stats (Teams, Projects, Tasks, Deadline) พร้อม Card สรุปทีมและโปรเจกต์ล่าสุด
- หน้า Teams — จัดการทีมและสมาชิก
- หน้า Projects — แสดงรายการโปรเจกต์ และ Board (Kanban) ภายในแต่ละโปรเจกต์
- หน้า Chat — ระบบแชทแบ่งตามโปรเจกต์ แสดงข้อความแบบ Bubble Chat
- หน้า Profile — แก้ไขข้อมูลส่วนตัวของผู้ใช้

บทที่ 3 การพัฒนา

3.1 Frontend Development

Frontend พัฒนาด้วย ReactJS 18 โดยใช้ Functional Components และ React Hooks ในทุกส่วน โครงสร้างการพัฒนา Frontend มีดังนี้

3.1.1 AuthContext และ State Management

ระบบจัดการสถานะผู้ใช้ด้วย React Context API ผ่าน AuthContext.jsx ซึ่งทำหน้าที่จัดเก็บข้อมูล User ใน State และมีฟังก์ชัน login(), register(), logout() พร้อม auto-restore session จาก localStorage Token เมื่อ App โหลด

3.1.2 Routing

ใช้ React Router DOM v6 จัดการ Routing โดยแบ่ง Route เป็น 2 ประเภท คือ PrivateRoute (ต้องล็อกอิน) และ PublicRoute (เข้าถึงได้โดยไม่ต้องล็อกอิน) หน้าที่ยอมรับได้แก่ /login, /dashboard, /teams, /projects, /chat, /profile

3.1.3 API Communication ด้วย Axios

ใช้ Axios Instance ที่กำหนด baseURL และ Request Interceptor สำหรับแนบ JWT Token อัตโนมัติใน Authorization Header ของทุก Request ทำให้ไม่ต้องจัดการ Token เข้าในแต่ละหน้า

3.2 Backend Development

Backend พัฒนาด้วย Node.js + Express.js ตามหลัก MVC Architecture โดยแยก Controller, Route, Model และ Middleware ออกจากกันอย่างชัดเจน

3.2.1 RESTful API Endpoints

ระบบมี API Endpoints ครอบคลุมทุกฟีเจอร์ โดยแบ่งตาม Resource ดังนี้

Method	Endpoint	หน้าที่
POST	/api/auth/register	สมัครสมาชิก
POST	/api/auth/login	เข้าสู่ระบบ รับ JWT Token
GET	/api/auth/me	ดึงข้อมูลผู้ใช้ปัจจุบัน
GET/POST	/api/teams	ดึงและสร้างทีม
GET/POST	/api/teams/:id/members	จัดการสมาชิกทีม
GET/POST/PATCH/DELETE	/api/projects	CRUD โปรเจกต์
GET/POST	/api/projects/:id/boards	จัดการ Board ในโปรเจกต์
GET/POST	/api/boards/:id/tasks	จัดการ Task ใน Board
PUT/DELETE	/api/tasks/:id	แก้ไขและลบ Task
GET/PUT/DELETE	/api/projects/:id/members	จัดการสมาชิกโปรเจกต์
GET	/api/projects/:id/chats	ดึง/สร้าง Chat Room
GET/POST	/api/chats/:id/messages	รับ-ส่งข้อความในแชท

3.2.2 Authentication Middleware

Middleware `authenticate.js` ทำหน้าที่ตรวจสอบ JWT Token ใน Authorization Header ของทุก Request ที่ต้องการสิทธิ์ หาก Token ถูกต้องจะ Decode `userId` แล้วเก็บไว้ใน `req.user` เพื่อให้ Controller ใช้งานต่อ หาก Token หมดอายุหรือไม่ถูกต้องจะส่ง HTTP 401 กลับ

3.2.3 Error Handling

ใช้ `express-async-errors` เพื่อ Catch Error จาก Async Functions อัตโนมัติ โดยไม่ต้องใช้ `try-catch` ทุกจุด พร้อม Global Error Handler Middleware ที่ส่ง Response Error ในรูปแบบ JSON ที่สม่ำเสมอ

3.3 เทคนิคต่าง ๆ ที่ใช้พัฒนา

- Password Hashing — ใช้ `bcryptjs` เข้ารหัส Password ด้วย Salt Factor 10 ก่อนจัดเก็บใน Database ผ่าน Mongoose Pre-save Hook
- JWT Authentication — ออก Token อายุ 7 วัน เมื่อ Login หรือ Register สำเร็จ
- Cascade Delete — เมื่อลบ Board จะลบ Task ทั้งหมดใน Board ด้วย (`Task.deleteMany`)
- Auto-create Chat Room — เมื่อเรียก `GET /api/projects/:id/chats` ระบบจะสร้าง Chat Room ให้อัตโนมัติหากยังไม่มี
- Role-based Permission — การแก้ไข Role สมาชิกต้องมีสิทธิ์ Owner เท่านั้น การแก้ไข Profile ทำได้เฉพาะเจ้าของ Account
- Populate — ใช้ Mongoose `.populate()` ดึงข้อมูล User ที่เกี่ยวข้องใน Response เช่น `assigneeId`, `senderId`
- Docker — ทั้ง Frontend และ Backend มี Dockerfile พร้อม Frontend ใช้ Nginx Serve Static Files

3.4 โครงสร้างของ Source Code

โครงสร้างโปรเจกต์แบ่งออกเป็น 2 ส่วนหลัก คือ frontend/ และ backend/ ดังนี้

```
PainnerAtlas/  
├─ backend/  
│   └─ app.js                # Entry point, Express setup,  
Routes  
│   └─ controllers/          # Business Logic (auth, team,  
project, board, task, chat, member, user)  
│   └─ models/               # Mongoose Schemas(index.js รวม 9  
Collections)  
│   └─ routes/               # Route Definitions แยกตาม  
Resource  
│   └─ middleware/           # authenticate.js, errorHandler.js  
│   └─ .env                  # Environment Variables (PORT,  
MONGO_URI, JWT_SECRET)  
└─ Dockerfile  
└─ frontend/  
    └─ src/  
        └─ App.jsx            # Router + Route Guards  
        └─ main.jsx           # React Entry  
        └─ api/axios.js        # Axios Instance + Interceptor  
        └─ context/           # AuthContext (Global State)  
        └─ components/        # Sidebar  
        └─ pages/             # Dashboard, Teams, Projects, Chat,  
Login, Profile  
    └─ index.html  
    └─ nginx.conf  
    └─ Dockerfile
```


บทที่ 4 การทดสอบและหน้าจอฟลัฟ

4.1 การทดสอบระบบ Authentication

ทดสอบการสมัครสมาชิกด้วยข้อมูลครบถ้วน ระบบสร้าง User ใน MongoDB พร้อมเข้ารหัส Password และส่ง JWT Token กลับมา จากนั้นทดสอบ Login ด้วย Email และ Password ที่ถูกต้องและไม่ถูกต้อง เพื่อตรวจสอบการตอบสนองของระบบ

ทดสอบการเรียก Protected Route โดยไม่มี Token และ Token หมดอายุ ระบบตอบกลับด้วย HTTP 401 พร้อมข้อความแสดงข้อผิดพลาดที่ชัดเจน

4.2 การทดสอบ CRUD Operations

ทดสอบการสร้าง แก้ไข และลบข้อมูลในทุก Resource ได้แก่ Team, Project, Board, Task และ Member โดยตรวจสอบ Response Status Code และข้อมูลที่ส่งกลับ รวมถึงทดสอบ Cascade Delete เมื่อลบ Board ว่า Task ที่เกี่ยวข้องถูกลบตามด้วย

4.3 การทดสอบ Chat System

ทดสอบการส่งข้อความในโปรเจกต์ โดยตรวจสอบว่า Chat Room ถูกสร้างอัตโนมัติเมื่อเรียกครั้งแรก ข้อความแสดงผลถูกต้องในรูปแบบ Bubble Chat พร้อมระบุชื่อผู้ส่งและเวลา รวมถึงตรวจสอบ Auto-scroll ไปยังข้อความล่าสุดเมื่อมีข้อความใหม่

4.4 หน้าจอฟลัฟ

สามารถแทรกภาพหน้าจอการทำงานของระบบในแต่ละหน้าได้ดังนี้ ได้แก่ หน้า Dashboard, Teams, Projects, Kanban Board, Chat และ Profile

บทที่ 5 ความท้าทายและอุปสรรคที่พบในระหว่างพัฒนา

5.1 การออกแบบ Database Schema

การออกแบบความสัมพันธ์ระหว่าง Collection ต่าง ๆ โดยเฉพาะการแยก ProjectMember ออกจาก TeamMember เพื่อให้รองรับ Role และ Display Name ที่แตกต่างกันในแต่ละโปรเจกต์ ต้องใช้เวลาในการวางแผนและทบทวนโครงสร้างหลายรอบก่อนลงมือพัฒนา

5.2 JWT Authentication และ Session Management

การจัดการ Token บน Frontend ให้ถูกต้อง โดยเฉพาะการ Restore Session เมื่อ Page Refresh และการ Handle กรณี Token หมดอายุกลางคันระหว่างใช้งาน ต้องออกแบบ AuthContext และ Axios Interceptor ให้ทำงานร่วมกันได้อย่างถูกต้อง

5.3 Nested Routes และการจัดการ State

การออกแบบ API Route ที่มีความ Nested ซับซ้อน เช่น

/api/projects/:projectId/boards/:boardId/tasks และการจัดการ State บน Frontend ให้อัปเดตแบบ Real-time เมื่อมีการเปลี่ยนแปลงข้อมูล โดยไม่ต้อง Reload หน้า

5.4 CORS Configuration

ในระยะแรกของการพัฒนาพบปัญหา CORS Error เมื่อ Frontend พยายามเรียก Backend API ซึ่งแก้ไขโดยการกำหนด CORS Policy บน Express ให้รองรับ Origin จาก Frontend URL และ Credential

บทที่ 6 สรุป

โครงการ PannerAtlas เป็นเว็บแอปพลิเคชันสำหรับบริหารจัดการโปรเจกต์และงานของทีม พัฒนาด้วยเทคโนโลยี Full-Stack ที่ประกอบด้วย ReactJS บน Frontend, Node.js + Express.js บน Backend และ MongoDB เป็นฐานข้อมูล

ระบบที่พัฒนาขึ้นรองรับการจัดการ Team, Project, Board (Sprint), Task และระบบ Chat ในโปรเจกต์ โดยมีระบบ Authentication ด้วย JWT และการควบคุมสิทธิ์ตาม Role ของสมาชิก การพัฒนาโครงการนี้ช่วยให้ทีมได้ฝึกทักษะการออกแบบและพัฒนา Full-Stack Web Application ตามมาตรฐาน MVC Architecture และ RESTful API อย่างครบถ้วน

แม้จะพบอุปสรรคต่าง ๆ ในระหว่างการพัฒนา แต่ทีมสามารถแก้ไขปัญหาได้โดยอาศัยความรู้ที่ได้รับจากการเรียนในชั้นเรียน ประกอบกับการค้นคว้าเพิ่มเติม ทำให้สามารถส่งมอบระบบที่ตรงตามวัตถุประสงค์ที่วางไว้ได้

บทที่ 7 ตารางสรุปการมีส่วนร่วม

ชื่อ-นามสกุล	รหัสนักศึกษา	งานที่ได้รับผิดชอบ
นายจุติณัฐ รัตนมงคลกุล	6609650228	models/index.js, register, login, getBoardsByProject, createBoard, deleteProject, getProjectStats
นายภูวฤทธิ์ เจริญพล	6609650590	createProject, getProject, getProjects, updateProject, fix: export error
นายรพีพันธ์ นะราช	6609650624	addTeamMember, removeTeamMember, updateTeamMember, getMembers, getMember, addMember, editMember, removeMember

นายศุภณัฐ ตั้งกิจวุฒิกุล	6609650665	get user and update user, getMessages and sendMessages, getProjectChat
นายสุทธิพนธ์ ประทุมทอง	6609650699	JWT Auth Middleware, getMe, errorHandler.js, getBoard, getTasksByBoard, updateBoard, deleteBoard
นายสุรบดี ผาสุข	6609650707	getMyTeams, createTeam, getTeamMembers, getTask, editTask, deleteTask, createTask, answers, video present